

시뮬레이션 재현 가이드 — 16-bit Bus 설계 프로젝트

보고서의 모든 수치를 다시 얻으려면 아래 테스트벤치들을 순서대로 재현하면 된다.

Linux(정식: Cadence Virtuoso + Spectre) 와 **Windows(원격 접속 또는 오픈소스 대체)** 두 경로를 모두 정리했다.

0. 재현 대상 테스트벤치 목록

#	테스트벤치	측정 대상	보고서 결과값 (기준)
TB-0a	Gate cap 추출 (slope matching)	C _{in} (unit)	4.1216 fF (W=3.3 μ m 기준)
TB-0b	Drain cap 추출 (drain에 신호 인가)	C _{d,n} / C _{d,p}	0.5930 fF / 1.405 fF
TB-0c	Mobility 비교 (t _r vs t _f)	β ratio $\rightarrow$ PN sizing	W _p :W _n = 2:1에서 균형
TB-0d	FO1/FO4 delay	τ , p _{inv}	15.04 ps / 27.40 ps $\rightarrow$ τ =4.121 ps, p _{inv} =2.650
TB-1	Source(+Destination) 설계 검증	t _{pd} , logic 정합성	368.4 ps (이론 378.6 ps, 오차 2.77%)
TB-2	Effective resistance	R _{eff} (unit n/p)	452.284 Ω / 469.991 Ω
TB-3	Non-repeated wire (π -model)	t _{pd}	이론 916.4 ps
TB-4	Repeated wire N=3/4/5 sweep	최적 N, t _{pd}	N=4, 762.2 ps
TB-5	Crosstalk 3-case	t _r 비교	409.9 / 795.4 / 2216 ps
TB-6	재설계 통합 (final)	t _{pd}	756.4 ps

공통 측정 규약 - V_{DD} = 1.1 V, 25 $^{\circ}$ C (과제 공통 조건) - Propagation delay t_{pd}: 입력·출력이 V_{DD}/2 = **550 mV**를 교차하는 시점 간 차이, rising/falling 각 5회 측정 후 평균 - Rise/fall time: **0.2 V_{DD} $\rightarrow$ 0.8 V_{DD} (220 mV $\rightarrow$ 880 mV)** - Unit inverter: L = 45 nm, W_n = 1.1 μ m, W_p = 2.2 μ m

1. Linux — Cadence Virtuoso + Spectre (정석 경로)

원 프로젝트가 수행된 방식 그대로다. 전제조건: **학교/연구실 리눅스 서버 계정, Cadence 라이선스, PDK**(수업은 45 nm 교육용 PDK, 튜터링 시연은 TSMC28 28HPC_RF). 셋 다 학교에서 제공받아야 하며 개인 PC에 임의 설치하는 라이선스 위반이다.

1.1 서버 접속

X11 포워딩으로 접속 (GUI를 로컬에 띄움)

`ssh -X 학번계정@서버주소`

예: `ssh -X user@ymsicl-x4.yonsei.ac.kr` (연구실 안내 기준)

원격 GUI가 느리면 VNC/X2Go/NoMachine 세션을 요청하는 것이 쾌적하다 (조교에게 확인).

1.2 환경 설정 및 실행

연구실/수업에서 배포한 `setup` 스크립트를 `source` (PDK 경로, 라이선스 서버 지정)

`source /PDK경로/setup.csh` # 또는 `.bashrc`에 안내된 명령

`cd ~/project && virtuoso &`

작업 디렉토리에 `cds.lib`(라이브러리 등록), `.cdsinit`이 있어야 한다. `cds.lib` 예:

`INCLUDE /PDK경로/cds.lib ; PDK 라이브러리 (tsmcN28 등)`

`DEFINE Tutoring ./Tutoring ; 내 작업 라이브러리`

1.3 라이브러리·회로 작성 순서 (튜터링 세션 절차 그대로)

1. **Library Manager** → File → New → Library → 이름 입력 → “Attach to an existing technology library” → PDK tech lib 선택 (예: tsmcN28)
2. New → Cell View → INV (schematic)
3. **Schematic Editor 단축키**: i 인스턴스 추가 → Browse에서 PDK 소자 선택 (예: nch_ulvt_mac, pch_ulvt_mac — LVT/HVT/ULVT는 V_{th} 옵션; low-power 어플리케이션이면 높은 V_{th}가 유리), w 와이어, l 와이어 라벨(같은 라벨끼리는 그리지 않아도 연결됨), q 속성 편집(W/L 변경: 예 W=1.1u/2.2u), c 복사, m 이동, r 회전, u 실행취소, f 화면 맞춤, x 체크&세이프
4. 소자 속성에서 **Width**를 변수로 둘 수도 있음 (예: C_{gate}처럼 sweep할 값)

5. 테스트벤치 셀 tb_INV: analogLib의 vdc(1.1 V), vpulse(입력 클럭), cap(부하), gnd 배치

1.4 ADE Explorer 시뮬레이션

1. Schematic에서 Launch → **ADE Explorer** → simulator: **spectre**
2. Analyses → tran, Stop time = 10n (충분한 toggle 관찰)
3. **Design Variables**: 예) C_gate 등록 → 값 0.1f:0.1f:1f 또는 리스트로 parametric sweep
4. Outputs → 관찰 노드(입출력) 체크 → **Netlist and Run** (녹색 ▶)
5. 파형 창(ViVA)에서:
 - 마커 a, b 두 개 찍으면 dx(시간차)/dy(전압차) 자동 표시 → rise time(220 → 880 mV), t_{pd}(550 mV 교차) 측정
 - 또는 Tools → **Calculator**의 내장 함수로 자동화:
 - delay(v("/IN") 0.55 1 "rising" v("/OUT") 0.55 1 "falling") → t_{pd}
 - riseTime(v("/OUT") 0.22 t 0.88 t nil "time") → t_r
6. Parametric 결과는 스위프 값별 파형이 겹쳐 표시됨 → slope 일치 지점 판별(TB-0a)

1.5 각 테스트벤치 구성 요점

- **TB-0a (gate cap)**: unit inverter 3단 체인 × 2벌. A벌 출력엔 측정 대상 MOS(gate만 노드에 연결, NMOS는 D/S/B → VSS, PMOS는 D/S/B → VDD), B벌 출력엔 analogLib cap(C_gate 변수). C_gate를 0.1f~1f 스위핑하여 두 출력 파형 slope가 일치하는 값 = gate cap.
- **TB-0b (drain cap)**: 같은 구조지만 **gate가 아니라 drain에 신호 노드 연결** (NMOS: G/S/B → VSS, drain → 노드).
- **TB-0c (mobility)**: 인버터 체인 끝에 cap, rising delay(220 → 880 mV)와 falling delay를 각 5회 측정 → W_p:W_n 비율을 바꿔가며 t_r=t_f가 되는 비율 탐색.
- **TB-0d (FO1/FO4)**: FO1은 동일 인버터 1개 부하, FO4는 4배 부하(또는 4개 병렬). 앞단에 input-shaping 인버터 3단 필수. $\tau = (t_{FO4} - t_{FO1})/3$, p_{inv} = t_{FO1}/τ - 1.
- **TB-2 (R_{eff})**: 소자를 linear 영역에 바이어스하고 DC sweep으로 I-V 기울기 측정 → 기울기 역수 = R_{eff}.

- **TB-3/4 (wire):** wire 1구간을 π -segment 3개(정확도 ~3%)로: 직렬 $R = R_w \cdot l/3$ 씩, 각 노드에 $C = C_w \cdot l/6 \dots (\pi \text{ 분배})$. Repeater는 driver(NOR2)와 동일 구동력의 인버터 2단. $N = 3/4/5$ 를 각각 스키매틱으로 만들어 비교.
- **TB-5 (crosstalk):** 동일한 wire 레더 2벌, 세그먼트마다 두 라인 사이에 $C_{adj} = 0.3 \text{ fF}/\mu\text{m} \times (l/\text{세그먼트수})$ 연결. Case1 동상 transition / Case2 인접선 고정 / Case3 역상 transition 3회 시뮬레이션.

1.6 배치 실행 (Ocean 스크립트, 선택)

GUI 없이 반복 측정을 자동화하려면:

```
; run.ocn
simulator('spectre)
design("./netlist/input.scs")
analysis('tran ?stop "10n")
desVar("C_gate" 0.2f)
run()
plot(v("/OUT"))
ocnPrint(?output "delay.txt" delay(v("/IN") 0.55 1 "rising" v("/OUT") 0.55 1
"rising"))
```

```
ocean -nograph -restore run.ocn
```

1.7 라이선스가 없는 경우 (Linux 오픈소스)

- **ngspice** (aptinstall 가능) + **PTM 45 nm 모델**(무료) — 아래 Windows 절의 netlist가 그대로 동작한다: `sudo apt install ngspice` 후 `ngspice tb0a_gatecap.cir`
- Xschem + ngspice (+ SkyWater SKY130 PDK)로 스키매틱 기반 재현도 가능. 단, 공정이 다르므로 **절대값은 달라지고 경향만 재현**된다.

2. Windows

Cadence는 Windows 네이티브를 지원하지 않는다. 두 가지 경로가 있다.

2.1 경로 A — 학교 리눅스 서버에 원격 접속 (권장, 결과 동일)

MobaXterm (무료, X11 서버 내장)이 가장 간단하다:

1. <https://mobaxterm.mobatek.net> → Home Edition 설치
2. Session → SSH → 호스트/계정 입력, **“X11-Forwarding” 체크 확인** (기본 on)

3. 접속 후 터미널에서 `source setup.csh && virtuoso &` → Virtuoso GUI가 Windows 화면에 뜬다
4. 이후 절차는 §1.3~1.6과 동일

대안: PuTTY + VcXsrv(X11 서버 별도 설치, PuTTY 설정에서 X11 forwarding enable), 또는 학교에서 VNC를 제공하면 TigerVNC Viewer.

팁: 과형 뷰어(ViVA)는 X11 원격에서 느낄 수 있다. MobaXterm 설정에서 압축을 켜거나, 서버에서 VNC 세션을 쓰면 쾌적하다.

2.2 경로 B — Windows 로컬에서 오픈소스로 재현 (ngspice + PTM 45 nm)

라이선스 없이 **회로 구조·측정 방법·경향성**을 완전히 재현할 수 있다. 공정 모델이 다르므로 절대값은 보고서와 다르게 나온다는 점만 유의(보고서와의 비교 목적이면 “경향 일치” 확인용).

설치

1. ngspice Windows 빌드 다운로드: <https://ngspice.sourceforge.io> → ngspice-xx_64.7z
압축 해제 (예: C:\ngspice)
2. PTM 45 nm 모델 다운로드: <https://ptm.asu.edu> → “45nm_HP” (high performance) → 45nm_HP.pm로 저장
 - 모델 카드의 소자명은 nmos/pmos, V_DD 1.0~1.1 V급 bulk CMOS
3. 작업 폴더에 netlist(.cir)와 모델 파일을 같이 두고:
C:\ngspice\bin\ngspice.exe tb0a_gatecap.cir

TB-0a — Gate capacitance (slope matching)

* tb0a_gatecap.cir – gate cap extraction by slope matching

```
.include 45nm_HP.pm
.param VDD=1.1
```

```
.subckt inv in out vdd vss
MP out in vdd vdd pmos W=2.2u L=45n
MN out in vss vss nmos W=1.1u L=45n
.ends
```

```
VDD vdd 0 {VDD}
VIN in 0 PULSE(0 {VDD} 1n 20p 20p 2n 4n)
```

* chain A: 3-inv shaping + DUT nmos (gate load) – D/S/B는 VSS

```
XA1 in a1 vdd 0 inv
XA2 a1 a2 vdd 0 inv
```

```
XA3 a2 outA vdd 0 inv
MDUT 0 outA 0 0 nmos W=1.1u L=45n
```

```
* chain B: 3-inv shaping + ideal cap
```

```
XB1 in b1 vdd 0 inv
XB2 b1 b2 vdd 0 inv
XB3 b2 outB vdd 0 inv
CL outB 0 0.2f
```

```
.tran 1p 12n
```

```
.control
```

```
foreach cg 0.1f 0.2f 0.3f 0.4f 0.5f 0.6f 0.7f 0.8f 0.9f 1.0f
  alter CL $cg
  run
  meas tran trA TRIG v(outA) VAL=0.22 RISE=2 TARG v(outA) VAL=0.88 RISE=2
  meas tran trB TRIG v(outB) VAL=0.22 RISE=2 TARG v(outB) VAL=0.88 RISE=2
end
.endc
.end
```

→ $trA \approx trB$ 가 되는 cg 가 gate capacitance. (pMOS는 MDUT를 pmos W=2.2u, D/S/B → vdd로 바꿔 반복)

TB-0b — Drain capacitance

TB-0a에서 DUT 연결만 변경: MDUT outA gnd_g 0 0 nmos W=1.1u L=45n 형태로 **drain**을 **outA**에, gate/source/bulk를 0(VSS)에 연결. (pMOS는 전부 vdd 기준)

TB-0d — FO1 / FO4 delay

```
* tb0d_fo4.cir
.include 45nm_HP.pm
.param VDD=1.1
.subckt inv in out vdd vss m=1
MP out in vdd vdd pmos W={2.2u*m} L=45n
MN out in vss vss nmos W={1.1u*m} L=45n
.ends
VDD vdd 0 {VDD}
VIN in 0 PULSE(0 {VDD} 1n 20p 20p 2n 4n)

* shaping 3단 → DUT → FO4 부하(4배 인버터) → 2차 부하(관찰 노드 왜곡 방지)
X1 in n1 vdd 0 inv
X2 n1 n2 vdd 0 inv
X3 n2 n3 vdd 0 inv
XD n3 n4 vdd 0 inv ; DUT
XL n4 n5 vdd 0 inv m=4 ; FO4 load
XL2 n5 n6 vdd 0 inv m=16
.tran 1p 12n
```

```
.meas tran tpdf TRIG v(n3) VAL=0.55 RISE=2 TARG v(n4) VAL=0.55 FALL=2
.meas tran tpdr TRIG v(n3) VAL=0.55 FALL=2 TARG v(n4) VAL=0.55 RISE=2
.end
```

FO1은 XL의 $m=4 \rightarrow m=1$. $\tau = (t_{FO4} - t_{FO1})/3$, $p_{inv} = t_{FO1}/\tau - 1$ 로 환산.

TB-3 — Non-repeated wire (π -model, 3 segments)

```
* tb3_wire_pi3.cir - l=200um, Rw=100ohm/um(총 20k), Cw=0.2fF/um(총 40fF)
.include 45nm_HP.pm
.param VDD=1.1
.subckt inv in out vdd vss m=1
MP out in vdd vdd pmos W={2.2u*m} L=45n
MN out in vss vss nmos W={1.1u*m} L=45n
.ends
VDD vdd 0 {VDD}
VIN in 0 PULSE(0 {VDD} 1n 20p 20p 4n 8n)
X1 in n1 vdd 0 inv
X2 n1 drv vdd 0 inv m=2 ; driver (보고서에선 NOR2 등가)
* 3-segment pi ladder: 세그먼트당 R=6.667k, 노드 cap 분배
6.67f/13.3f/13.3f/6.67f
Rw1 drv w1 6.667k
Rw2 w1 w2 6.667k
Rw3 w2 rcv 6.667k
Cp0 drv 0 6.667f
Cp1 w1 0 13.333f
Cp2 w2 0 13.333f
Cp3 rcv 0 6.667f
Crcv rcv 0 7.226f ; receiver input cap
.tran 1p 20n
.meas tran tpd TRIG v(n1) VAL=0.55 FALL=2 TARG v(rcv) VAL=0.55 RISE=2
.end
```

TB-4 — Repeated wire (N=4)

$l/4 = 50 \mu\text{m}$ 구간 4개, 구간마다 위 래더를 $R=5 \text{ k}/C=10 \text{ fF}$ 규모로 축소하고 사이에 repeater(인버터 2단, driver와 동일 구동력) 삽입. $N=3/5$ 버전을 만들어 t_{pd} 비교 $\rightarrow$ 최소 N 확인.

TB-5 — Crosstalk

```
* 두 개의 동일 wire 래더 lineA/lineB, 세그먼트 노드마다 coupling cap
Cc1 a_w1 b_w1 20f ; Cadj=0.3fF/um x 200um/3seg  $\approx 20\text{f}$  per node
Cc2 a_w2 b_w2 20f
Cc3 a_rcv b_rcv 20f
* Case1: VB = VA와 동상 / Case2: VB=DC 0 / Case3: VB 역상 펄스
.meas tran trA TRIG v(a_rcv) VAL=0.22 RISE=1 TARG v(a_rcv) VAL=0.88 RISE=1
```

세 케이스의 t_rA 비가 대략 $C_{gnd} : C_{gnd}+C_{adj} : C_{gnd}+2C_{adj}$ 를 따르는지 확인
(보고서: 1 : 1.94 : 5.41).

TB-1/TB-6 — Source/Destination 로직

NAND2/NOR2/NAND4/NAND3를 서브서킷으로 정의하고(표 4.3, 4.9의 W 값 반영), 8-stage 경로를 조립 → 550 mV 교차 t_{pd} 측정 + 진리표 토글 검증(표 2.5의 a0/a1/s0b 파형 재현).

2.3 경로 B' — LTspice를 선호하는 경우

GUI 파형 확인이 편하다. LTspice 설치 → `.include 45nm_HP.pm` → 위 netlist를 `.asc` 없이 netlist 그대로 열어도 실행됨. `.step param CG 0.1f 1f 0.1f`(LTspice는 `.step` 지원)로 slope-matching이 한 번에 된다. 측정은 `.meas` 동일.

2.4 경로 C — WSL2 (Windows 안의 Linux)

```
wsl --install -d Ubuntu
```

```
sudo apt update && sudo apt install ngspice xschem
```

이후 §1.7과 동일. Xschem으로 스키매틱을 그리고 ngspice로 돌리는 GUI 흐름도 가능하다.

3. 재현 검증 체크리스트

확인 항목	기대 결과 (Cadence+수업 PDK 기준)	오픈소스(PTM)에서의 기대
slope matching cap $\approx$ gate cap	4.1216 fF @ 3.3 μ m	값은 다르나 W 에 비례
$t_r = t_f$ 되는 PN ratio	$\approx 2:1$	1.5~2.5:1 (모델별 상이)
FO4/FO1 → τ , p_{inv}	4.121 ps / 2.650	수 ps대, p_{inv} 1~3
W 10배 증가 시 t_{pd}	거의 불변 (self-loading)	동일 경향
delay vs load cap	parasitic 때문에 비선형 오프셋	동일 경향
repeater N sweep	$N=4$ 최소, ~ 250 ps 개선	최소점 존재 (N 값은 다를 수 있음)
crosstalk 3-case	1 : 1.94 : 5.41	1 : ~ 2 : ~ 5 근방

확인 항목	기대 결과 (Cadence+수업 PDK 기준)	오픈소스(PTM)에서의 기대
이론(logical effort) vs 시뮬	오차 ~3% (소자만) / ~15% (wire 포함)	유사 수준

4. 자주 막히는 지점

1. **X11 GUI가 안 뜸** → MobaXterm이면 세션 설정 X11-Forwarding 확인, `echo $DISPLAY`가 비어 있으면 재접속. 서버 측 `xauth` 문제일 수 있으니 조교 문의.
2. **ngspice에서 .step 오류** → ngspice는 .step 미지원. .control 블록의 `foreach + alter`를 사용 (위 예시).
3. **PTM 모델 소자명 불일치** → 모델 카드 안의 `.model NMOS nmos ...` 이름 확인 후 netlist의 M 소자 모델명을 맞추는 것.
4. **측정이 안 잡힘 (meas failed)** → TRIG/TARG의 RISE/FALL 차수가 파형에 실제로 존재하는지 확인. 펄스 지연(1n)보다 tran 구간이 충분히 긴지 확인.
5. **파형이 완전히 정착하기 전 측정** → 첫 사이클은 초기 조건 영향이 있으므로 2번째 이후 `edge(RISE=2 등)`로 측정 (보고서도 5회 평균 사용).
6. **Virtuoso에서 같은 라벨인데 연결 안 됨** → 라벨은 wire 위에 정확히 얹혀야 한다. Check and Save(x)에서 floating net 경고 확인.

작성: 사진 속 튜터링 세션(Virtuoso 절차)과 보고서 측정 규약을 기반으로 재구성